KIRTA: Knowledge, Inclusion & Real-Time Assistance*

Maksym Bulat, Yevhenii Demianchuk, Yehor Donin
Slovak University of Technology in Bratislava
Faculty of Informatics and Information Technologies
xbulat@stuba.sk, xdemianchuk@stuba.sk, xdonin@stuba.sk

October 30, 2026

Annotation

The essence of the project

KIRTA is a voice-first daily-life AI companion for blind and low-vision people.

Target audience

More than 1 billion people experience sight loss; 43 million of them are blind [1]. KIRTA's primary target audience is blind and low-vision people. However, later expansion into people with other disabilities is possible.

Problem

Nowadays, people depend on smartphones for navigation, communication, shopping, studying, working, banking, entertainment, and other mundane tasks. People with blindness are no exception; for them, smartphones serve as a lifeline [2], replacing their sight. Yet using a phone requires them to hold it, orient themselves on the touchscreen, and ask other people for help when needed. Existing apps mostly work on request; for example, reading text from a photo/screen one at a time or helping with navigation. They also struggle with understanding users' intent [3].

Solution

Instead of holding the phone all the time, the phone is secured by a case with a strap. Its camera acts as the user's eyes, analyzing the surroundings with AI. Then the information is narrated to the user's headphones. Most of the time, the app is silent, breaking the silence only in critical moments, when the user is in danger, or on request. KIRTA is also expected to help with

*Semester project in Metódy inžinierskej práce, academic year 2026/27, supervisor: Ing. Vladimír Mlynarovič, PhD.

navigation, calls, messages, and alerts. More functions are to come later; for example, ordering goods and food, consent-based face recognition [4–6], tourist guidance, and integration into desktop operating systems. Security comes first, so everything involving biometric data and hazard detection will run locally, and payments are always confirmed by the user in the service's own app. Other features use the cloud when the internet is available.

Monetization

Essential and local functions will be free. However, the price of the premium subscription will depend on the support of partners (banks, retailers, transit) and the state [7].

List of functions

Video-to-audio translation

Navigation

The navigation design for KIRTA combines GPS with local camera analysis. GPS provides the general position on the map and supports the route to a destination. The local AI follows the camera feed and identifies what is happening near the person. These sections describe a preliminary design, with implementation and testing to follow.

A person may be on the correct route and still face a blocked pavement, stairs or a vehicle approaching from the side. The navigation feature needs to deal with that part of the journey too. The phone will be attached to a strap, with its camera facing the surroundings, leaving the person's hands free.

GPS.gov reports typical smartphone accuracy within about 4.9 metres under open sky. Accuracy gets worse near buildings, bridges and trees [8]. GPS gives the app a starting position, while the camera supplies a view of the immediate surroundings. Another goal is to use visual information to refine the position estimate as the person moves. Matching what the camera sees to a particular place on the map remains an open technical question.

Hazard detection is planned to run locally on the phone, analysing the camera feed throughout the walk. The recognition tasks include stairs, obstacles along the path and moving vehicles. With vehicles, movement matters as much as recognising the object: the system needs to assess whether it is approaching the user's path. Evaluation will measure detection errors and the delay between a hazard appearing and a warning being spoken.

In the study of Be My AI by Xie and colleagues, participants described stopping to take pictures and checking whether they had captured the information needed for navigation. Some descriptions also lacked details about nearby obstacles [3]. These findings are relevant to KIRTA's use of a camera feed that updates as the user moves.

NavCog offers another useful reference. It used Bluetooth beacons installed in the environment and was tested with six blind participants on a university campus [9]. Our design centres on the camera carried by the user. Soundscape is relevant to the audio side: it explored spatial sound for nearby places and

a chosen destination [10]. These projects give us examples of both location support and ways to communicate directions.

For everyday use, KIRTA should stay quiet until there is useful information to pass on. A possible hazard takes priority; a fuller description of the surroundings can be requested. The assistance is intended to work alongside the person's mobility skills and aids, including a white cane or guide dog.

Narration of on-screen activity

When a new SMS arrives, KIRTA should let the user know and offer to read it. This is the simplest example of screen narration in the project. The same feature covers incoming calls, notifications and changes in the open app. AI is intended to interpret this information and produce a short explanation that can be spoken through the headphones.

The explanation needs to fit what the person is doing. After pressing a button, the useful information is usually the result: whether the action succeeded, whether the app is still waiting or whether an error appeared. On opening an unfamiliar screen, a brief description of its contents and available actions may be more useful. These situations call for different amounts of detail. Repeating the whole screen after every change would make even a simple task tiring.

Android provides two possible sources for this feature. Accessibility services can inspect available interface elements and receive events such as clicks and changes of focus [11]. A notification listener service receives updates when notifications are posted or removed [12]. Access depends on the required configuration and permissions. In the initial Android design, these mechanisms would supply information for the AI to explain. Image-based recognition is a further option for screens where the exposed text and controls are insufficient.

W3C guidance helps specify the information worth keeping in that explanation. An accessible control has a name and role, together with relevant state information. A checkbox needs its label and whether it is selected [13]. Status messages describe progress, errors and action results without requiring the user to move focus to the message [14]. A meaningful focus order also helps people follow web content [15]. We use these web accessibility principles as a reference for the spoken interaction in KIRTA.

Our proposed interaction gives the user control over longer descriptions and private messages. A short notification can be followed by a request to read more, repeat it or stop speaking. The account of AI errors in Xie and colleagues' study is relevant here too: generated explanations can include details that the input does not support [3]. Screen narration therefore needs to stay grounded in the available interface information. Confirmation that a message was sent, for example, must come from the app's reported state.

Phone narration will share the audio output with navigation warnings. The proposed design gives a possible hazard outside priority over an ordinary notification and avoids repeated announcements of unchanged information. Evaluation should include tasks such as noticing a new message and understanding a change on the screen. The records should include missed updates, incorrect descriptions and unnecessary interruptions.

Friend

Voice command recognition

Voice commands will be one of the main ways to use KIRTA. For a blind person, it is often easier to say what they need than to look for a button on a touchscreen. The user could say "Describe what is in front of me", "Start navigation" or "Call Anna". Our goal is to make these commands short and easy to remember. The application should not require the user to say a long, exact sentence every time.

To recognize speech, we could use the Android `SpeechRecognizer` API. It receives spoken words and returns text which can then be checked by the application [16]. At first, KIRTA could support about ten basic commands. For example, if the recognized text contains "call" and a contact name, the application would understand that the user wants to make a phone call. Other commands could request a description of the camera image, repeat the last answer or stop the audio output.

Recognition will not always work perfectly. Traffic, other people talking and different accents can make words harder to understand. If KIRTA is not sure what the user said, it should ask for the command again. Some Android phones support on-device speech recognition, but it is not available for every phone or language [16]. We would prefer local processing when possible because it can also help with privacy. For testing, we could try the same ten commands in a quiet room and outside, and count how many times they are understood correctly. We could also ask a few classmates to try the same commands. It would help us see whether the words are clear enough for people who did not write the program.

Voice-controlled calls and messages

Another useful function is starting calls and writing messages without searching through the phone menus. For example, a user could say "Call Mom". KIRTA would search for the contact and repeat the name so the user knows who was selected. The same idea works for messages: "Send a message to Peter" would start a short conversation where KIRTA asks what the message should say. The user could then dictate the text normally.

Before doing anything important, KIRTA should confirm what it understood. It could say "Message to Peter: I will be late. Is that correct?" and wait for "yes" or "no". This is especially useful when two contacts have similar names or when speech recognition makes a mistake. The user should also be able to cancel the action at any point.

For a first version, we do not need to build our own phone or messaging service. Android provides `Intent` actions for opening the phone dialer and preparing an SMS message [17]. The normal Android application would finish the call or send the message. This also means that some final steps may still depend on the accessibility functions of the phone, such as TalkBack. Later, we could look into other messaging applications, but starting with calls and SMS is a more realistic plan.

One problem could be finding the correct contact. People often save more than one number under the same name, for example a personal and a work number. In this case, KIRTA should ask which number the user wants. We

would also try messages with short names, longer names and names that sound similar. The aim is not to make the phone call faster at any cost, but to make the action understandable.

Programming and technologies

We plan to develop KIRTA as an Android application. We would use Kotlin because it is supported for Android development and can be used in Android Studio [18]. We chose Android mainly because smartphones already have a camera, microphone, speaker and support for Bluetooth headphones. We can use these parts instead of creating separate hardware for the first prototype.

The application will need a few basic parts that work together. The first part listens to the user and recognizes commands. The second part decides what the command means and which action to start. A third part reads answers aloud using Android `TextToSpeech` [19]. For example, after hearing "What is in front of me?", the program would request information from the camera part and then speak the result. The sound could be played through connected Bluetooth headphones using the phone's normal audio settings.

For camera access, we could use the CameraX library. It is made for Android camera applications and could later provide images to an existing recognition model [20]. We do not plan to train a new AI model from zero for this stage. Our first goal would be to connect simple functions and make them work reliably. The other team members could then work on navigation and recognizing possible dangers, while the voice interface would connect these features to the user.

We would test the first version with a small set of commands, different contact names and simple spoken messages. We also need to check permissions for the microphone, camera and contacts. It is possible that the app will work differently on different Android devices, so testing on a real phone will be important. At this stage, KIRTA is still a project idea, and the exact technologies may change after the first experiments.

A simple development plan would start with one Android screen and a button for recording a command. Next, we could add spoken answers and a few actions, such as opening the dialer. Only after these functions work would we connect the camera and other features from the team. It is easier to find problems step by step than to build everything at once.

Literature

[1] International Agency for the Prevention of Blindness. Global data. IAPB Vision Atlas, 2026.

[2] S. S. Senjam and S. A. Primo. Challenges and enablers for smartphone use by persons with vision loss during the COVID-19 pandemic: A report of two case studies. *Frontiers in Public Health*, 10, 2022.

[3] J. Xie, R. Yu, H. Zhang, S. M. Billah, S. Lee, and J. M. Carroll. Beyond visual perception: Insights from smartphone interaction of visually impaired users with large multimodal models. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM, 2025.

[4] European Parliament and Council of the European Union. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of

personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). Official Journal of the European Union, L 119, 4.5.2016, pp. 1–88, 2016.

[5] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L, 2024/1689, 12.7.2024, 2024.

[6] T. Akter, T. Ahmed, A. Kapadia, and M. Swaminathan. Shared privacy concerns of the visually impaired and sighted bystanders with camera-based assistive technologies. *ACM Transactions on Accessible Computing*, 15:1–33, 2022.

[7] European Parliament and Council of the European Union. Directive (EU) 2019/882 on the accessibility requirements for products and services (European Accessibility Act). Official Journal of the European Union, L 151, 7.6.2019, pp. 70–115, 2019.

[8] GPS.gov. GPS Accuracy. `https://www.gps.gov/gps-accuracy`. Accessed 30 October 2026.

[9] Dragan Ahmetovic, Cole Gleason, Chengxiong Ruan, Kris Kitani, Hironobu Takagi, and Chieko Asakawa. NavCog: A Navigational Cognitive Assistant for the Blind. In *Proceedings of the 18th International Conference on Human-Computer Interaction with Mobile Devices and Services*, 2016.

[10] Microsoft Research. Microsoft Soundscape. `https://www.microsoft.com/en-us/research/product/soundscape/`. Accessed 30 October 2026.

[11] Android Developers. Create an accessibility service. `https://developer.android.com/guide/topics/ui/accessibility/service`. Accessed 30 October 2026.

[12] Android Developers. NotificationListenerService — Android API reference. `https://developer.android.com/reference/android/service/notification/NotificationListenerService`. Accessed 30 October 2026.

[13] World Wide Web Consortium. Understanding SC 4.1.2: Name, role, value. `https://www.w3.org/WAI/WCAG22/Understanding/name-role-value.html`. Accessed 30 October 2026.

[14] World Wide Web Consortium. Understanding SC 4.1.3: Status messages. `https://www.w3.org/WAI/WCAG22/Understanding/status-messages.html`. Accessed 30 October 2026.

[15] World Wide Web Consortium. Understanding SC 2.4.3: Focus order. `https://www.w3.org/WAI/WCAG22/Understanding/focus-order.html`. Accessed 30 October 2026.

[16] Android Developers. SpeechRecognizer — Android API reference. `https://developer.android.com/reference/android/speech/SpeechRecognizer`. Accessed 29 October 2026.

[17] Android Developers. Common intents — phone and text messaging. `https://developer.android.com/guide/components/intents-common`. Accessed 29 October 2026.

[18] Android Developers. Kotlin and Android. `https://developer.android.com/kotlin`. Accessed 29 October 2026.

[19] Android Developers. TextToSpeech — Android API reference. `https://developer.android.com/reference/android/speech/tts/TextToSpeech`. Accessed 29 October 2026.

[20] Android Developers. CameraX overview. `https://developer.android.com/media/camera/camerax`. Accessed 29 October 2026.